



Parul[®]
University

NAAC **A++**
ACCREDITED UNIVERSITY

FACULTY OF ENGINEERING
AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

JAVA PROGRAMMING LABORATORY
(03010503PC04)

III SEMESTER

CSE AIDS Department



Laboratory Manual
Session 2026-27

CERTIFICATE

This is to certify that

Mr / Ms _____ with enrollment no.

_____ has successfully completed
his/her laboratory experiments in the Java Programming Laboratory
(03010503PC04) from the department of **COMPUTER SCIENCE &
ENGINEERING AIDS** during the academic year 2026-2027.



Date of Submission:.....

Staff In charge:.....

Head Of Department:.....



TABLE OF CONTENT

Sr. No	Experiment Title	Date of Start	Date of Completion	Marks (out of 10)	Sign
1.	Write a Java program to display “Hello World” and demonstrate the Java program structure and demonstrate use of variables, data types, and type casting.				
2.	Write a Java program to perform arithmetic, relational, and logical operations, bitwise and other operators.				
3.	Write a Java program using below conditional statements. a) Even or Odd using if.. else statement. b) Roots of Quadratic Equation using else if ladder. c) Largest of three numbers using nested if else. d) Find out the week day using switch statement.				
4.	Write a Java program to demonstrate looping constructs: a) Reverse of a number using while loop. b) Prime number using do while loop. c) nth term of fibonacci sequence using for loop.				
5.	Write a Java program to perform methods and example on recursion.				
6.	Write a Java program to perform operations on one-dimensional and multi-dimensional arrays.				
7.	Write a Java program to demonstrate 1D - Arrays & 2-D Arrays. a) Maximum value and Second Maximum value without duplicates. b) Sort the names in Ascending Order. c) Addition of two matrix. d) 3x3 Matrix Multiplication				
8.	Write a Java program to demonstrate string handling using String, StringBuffer, and StringBuilder.				
9.	Write a Java program to check the word is palindrome or not.				
10.	Write a Java program to create a class and object and demonstrate constructors.				
11.	Write a Java program to demonstrate encapsulation using access specifiers.				



12.	Write a Java program to demonstrate inheritance: a) Single Inheritance. b) MultiLevel Inheritance. c) Hierarchical Inheritance. d) Hybrid Inheritance.				
13.	Write a Java program to demonstrate use of the this and super keywords				
14.	Write a Java program to demonstrate polymorphism (method overloading and overriding).				
15.	Write a Java program to implement abstraction using abstract classes and interfaces(Multiple Inheritance).				
16.	Write a Java program to demonstrate exception handling using try-catch-finally blocks.				
17.	Write a Java program to demonstrate exception handling using try-catch-finally blocks.				
18.	Write a Java program to implement List interface using ArrayList and LinkedList.				
19.	Write a Java program to implement Set interface using HashSet and TreeSet				
20.	Write a Java program to implement Queue interface using Priority Queue and Deque.				
21.	Write a Java program to implement map interface using HashMap and LinkedHashMap				
22.	Write a Java program to demonstrate multithreading using Thread class and Runnable interface				



PRACTICAL NO: 01

AIM:

Write a Java program to display “Hello World” and demonstrate the Java program structure and demonstrate use of variables, data types, and type casting.

THEORY:

Java is a high-level, object-oriented programming language. Every Java program is written inside a class, and execution starts from the main() method.

Java Program Structure:

1. **Class Declaration** – Defines the class.
2. **Main Method** – Entry point of the program.
3. **Statements** – Instructions executed by the program.

Variables:

Variables are named memory locations used to store data. Example: `int age = 20;`

Data Types:

- `int` – Integer values
- `float` – Decimal values
- `double` – Large decimal values
- `char` – Single character
- `boolean` – true or false
- `String` – Sequence of characters

Type Casting:

Type casting is the process of converting one data type into another.

- **Implicit (Widening):** Automatically converts a smaller data type to a larger one.
- **Explicit (Narrowing):** Manually converts a larger data type to a smaller one using casting.

CODE SNIPPET:



```
public class HelloWorld {  
  
public static void main(String[] args) {  
  
    // Display Hello World  
    System.out.println("Hello World");  
  
    // Variables and Data Types  
    int age = 20;  
    float height = 5.8f;  
    double salary = 45000.75;  
    char grade = 'A';  
    boolean isStudent = true;  
  
    // Display variable values  
    System.out.println("Age: " + age);  
    System.out.println("Height: " + height);  
    System.out.println("Salary: " + salary);  
    System.out.println("Grade: " + grade);  
    System.out.println("Student: " + isStudent);  
  
    // Implicit Type Casting (Widening)  
    int num = 100;  
    double d = num;  
    System.out.println("Implicit Type Casting (int to double): " + d);  
  
    // Explicit Type Casting (Narrowing)  
    double value = 99.99;  
    int newValue = (int) value;  
    System.out.println("Explicit Type Casting (double to int): " + newValue);  
}  
}
```



OUTPUT:

```
PROBLEMS 44 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\OOPJ LAB> javac Practical1.java
PS D:\OOPJ LAB> java Practical1
Hello World
50.0
20
30
N
22
77.7
true
20000
10.0
77
PS D:\OOPJ LAB> 
```



PRACTICAL NO: 02

AIM:

write a Java program to perform arithmetic, relational, and logical operations, bitwise and other operators.

THEORY:

Types of Operators

1. Arithmetic Operators

- Perform mathematical calculations.
- Operators: +, -, *, /, %, ++, --

2. Relational Operators

- Compare two values and return true or false.
- Operators: ==, !=, >, <, >=, <=

3. Logical Operators

- Combine or negate conditions.
- Operators: && (AND), || (OR), ! (NOT)

4. Bitwise Operators

- Perform operations on binary values.
- Operators: &, |, ^, ~, <<, >>, >>>

5. Assignment Operators

- Assign values to variables and simplify expressions.
- Operators: =, +=, -=, *=, /=, %=

6. Unary Operators

- Operate on a single operand.
- Operators: +, -, ++, --, !, ~

7. Ternary Operator

- A shorthand form of the if-else statement.
- Syntax:

condition ? value1 : value2;



Code Snippet:

```
class Practical2
```

```
{ public static void main(String[] args)
```

```
int a = 10;//Initialization
```

```
int b = 5
```

```
// Arithmetic Operators is defined as the operators that are used to perform mathematical operations on operands.
```

```
System.out.println("Arithmetic Operators:");
```

```
System.out.println("Addition = " + (a + b));//15
```

```
System.out.println("Subtraction = " + (a - b));
```

```
System.out.println("Multiplication = " + (a*b));
```

```
System.out.println("Division"+(a/b));
```

```
System.out.println("Modulus = " + (a%b));
```

```
// Relational Operators is defined as the operators that are used to compare two values.
```

```
// The result of a relational operator is always a boolean value (true or false).
```

```
System.out.println("\nRelational Operators: ");
```

```
System.out.println("a > b =" + (a > b));
```

```
System.out.println("a < b =" + (a < b));
```

```
System.out.println("a==b=" + (a== b));
```

```
System.out.println("a != b =" + (a != b));
```

```
System.out.println("a==b=" + (a >= b));
```

```
System.out.println("a b" + (a <= b));
```

```
// Logical Operators are used to perform logical operations on boolean values.
```

```
// They are used to combine multiple boolean expressions and return a single boolean result.
```

```
boolean x = true;//default value for boolean data type is false, so we have to assign true or false value to boolean data type
```

```
variable
```

```
boolean y false;
```

```
System.out.println("\nLogical Operators: ");
```

```
System.out.println("x && y =" + (x && y));
```

```
System.out.println("x || y=" + (x || y));// Logical or
```

```
System.out.println("!x =" + (!x));
```

```
System.out.println("\nBitwise Operators:"); System.out.println("a & b =" + (a & b));
```

```
System.out.println("a | b =" + (a | b));
```

```
System.out.println("a^b="+(a^b));
```

```
System.out.println("-a =" + (-a));
```

```
// Assignment Operators are used to assign values to variables.
```

```
// They are used to perform an operation and assign the result to a variable in a single step.
```

```
int c = 20
```

```
c+= 5; System.out.println("\nAssignment Operator (c+= 5): " + c);
```

```
// Increment and Decrement Operators are used to increase or decrease the value of a variable by 1.
```

```
int d 10;//initialization of variable d with value 10
```

```
System.out.println("\nIncrement and Decrement:");
```

```
System.out.println("d++" + (d++));
```

```
System.out.println("After d++=" + d);
```

```
System.out.println("--d=" + (--d));
```

```
// Conditional (Ternary) Operator are used to evaluate a boolean expression and return one of two values based on the result of the expression.
```

```
String result = (a > b)? "a is greater": "b is greater";
```

```
//Ternary operator is a shorthand for if-else statement,
```

```
// it takes three operands: a condition, a result for true, and a result for false.
```

```
System.out.println("\nTernary Operator:");
```

```
System.out.println(result);
```

```
} }
```



OUTPUT:

```
[Running] cd "c:\Users\vishal_gohil\Desktop\vishal java file\" && javac Practical2.java && java Practical2
Arithmetic Operators:
Addition = 15
Subtraction = 5
Multiplication = 50
Division = 2
Modulus = 0

Relational Operators:
a > b = true
a < b = false
a == b = false
a != b = true
a >= b = true
a <= b = false

Logical Operators:
x && y = false
x || y = true
!x = false

Bitwise Operators:
a & b = 0
a | b = 15
a ^ b = 15
~a = -11

Assignment Operator (c += 5): 25

Increment and Decrement:
d++ = 10
After d++ = 11
--d = 10

Ternary Operator:
a is greater

[Done] exited with code=0 in 0.907 seconds
```



PRACTICAL NO: 03

AIM:

Write a Java program using below conditional statements. a) Even or Odd using if.. else statement. b) Roots of Quadratic Equation using else if ladder. c) Largest of three numbers using nested if else. d) Find out the week day using switch statement.

THEORY:

Conditional statements are used to make decisions in a Java program. They execute different blocks of code based on whether a condition is true or false.

Types of Conditional Statements

1. if-else Statement

Used to execute one block of code if the condition is true and another block if it is false.

Application: Finding whether a number is Even or Odd.

2. else-if Ladder

Used to check multiple conditions one after another.

The first true condition is executed, and the remaining conditions are skipped.

Application: Finding the Roots of a Quadratic Equation based on the discriminant.

3. Nested if-else

An if-else statement placed inside another if-else statement.

Used when multiple conditions depend on each other.

Application: Finding the Largest of Three Numbers.

4. switch Statement

Used to select one block of code from multiple choices.

Works with values such as int, char, String, and enum.

Application: Displaying the Week Day based on the user's choice.

Advantages

Makes decision-making easy.

Improves program readability.

Reduces code complexity.

Supports multiple conditions efficiently.

Useful in menu-driven programs.

Applications

Even or Odd checking.

Finding roots of quadratic equations.

Finding the largest number.

Displaying weekdays and menu options.

Input validation and decision-making.



Code Snippet:

```
import java.util.Scanner;

public class EvenOdd {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        if (num % 2 == 0) {
            System.out.println(num + " is Even.");
        } else {
            System.out.println(num + " is Odd.");
        }

        sc.close();
    }
}
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\vishal gohil\Desktop\vishal java file> java EvenOdd.java
Enter a number: 5
5 is Odd.
PS C:\Users\vishal gohil\Desktop\vishal java file> 
```

// Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).

```
import java.util.Scanner;

public class QuadraticEquation {
    public QuadraticEquation() {
    }

    public static void main(String[] var0) {
        Scanner var1 = new Scanner(System.in);
        System.out.print("Enter value of a: ");
        double var2 = var1.nextDouble();
        System.out.print("Enter value of b: ");
        double var4 = var1.nextDouble();
        System.out.print("Enter value of c: ");
        double var6 = var1.nextDouble();
        double var8 = var4 * var4 - (double)4.0F * var2 * var6;
        if (var8 > (double)0.0F) {
            double var10 = (-var4 + Math.sqrt(var8)) / ((double)2.0F * var2);
            double var12 = (-var4 - Math.sqrt(var8)) / ((double)2.0F * var2);
            System.out.println("Two distinct real roots:");
            System.out.println("Root 1 = " + var10);
            System.out.println("Root 2 = " + var12);
        } else if (var8 == (double)0.0F) {
            double var14 = -var4 / ((double)2.0F * var2);
            System.out.println("Two equal real roots:");
            System.out.println("Root = " + var14);
        } else {
            System.out.println("The roots are imaginary (complex).");
        }

        var1.close();
    }
}
```



OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\vishal gohil\Desktop\vishal java file> java QuadraticEquation
Enter value of a: 2
Enter value of b: -4
Enter value of c: 1
Two distinct real roots:
Root 1 = 1.7071067811865475
Root 2 = 0.2928932188134524
PS C:\Users\vishal gohil\Desktop\vishal java file> |
```

Code Snippet:

```
import java.util.Scanner;

class LargestNumber {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int a, b, c;
        System.out.print("Enter three numbers: ");
        a = sc.nextInt();
        b = sc.nextInt();
        c = sc.nextInt();

        if (a > b) {
            if (a > c)
                System.out.println(a + " is Largest.");
            else
                System.out.println(c + " is Largest.");
        } else {
            if (b > c)
                System.out.println(b + " is Largest.");
            else
                System.out.println(c + " is Largest.");
        }
    }
}
```

OUTPUT:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\vishal gohil\Desktop\vishal java file> java LargestNum
Enter three numbers: 6
3
5
6 is Largest.
PS C:\Users\vishal gohil\Desktop\vishal java file> |
```



Code Snippet:

```
import java.util.Scanner;

class WeekDay {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter day number (1-7): ");
        int day = sc.nextInt();

        switch(day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");
                break;
            case 6:
                System.out.println("Saturday");
                break;
            case 7:
                System.out.println("Sunday");
                break;
            default:
                System.out.println("Invalid Day Number");
        }
    }
}
```

OUTPUT:

```
PS C:\Users\vishal gohil\Desktop\vishal java file> java Week.java
Enter day number (1-7): 5
Friday
PS C:\Users\vishal gohil\Desktop\vishal java file> █
```



PRACTICAL NO: 04

AIM:

Write a java program to demonstrate looping constructs: a) Reverse of number using while loop. b) Prime number using do while loop. c) nth term of fibonacci sequence using for loop.

THEORY:

1] Loops are control statements in Java that allow a block of code to be executed repeatedly until a specified condition is met. They reduce code repetition, improve efficiency, and make programs easier to understand and maintain. Java provides three main types of loops: while, do-while, and for.

1. While Loop

The while loop is an entry-controlled loop, which means the condition is checked before executing the loop body. If the condition is false initially, the loop will not execute even once. It is mainly used when the number of iterations is not known beforehand. In this practical, the while loop is used to reverse a number by repeatedly extracting its last digit using the modulus (%) operator and building the reversed number.

Syntax:

```
while(condition)
{
    // statements
}
```

2. Do-While Loop

The do-while loop is an exit-controlled loop. The statements inside the loop are executed first, and then the condition is checked. Therefore, the loop executes at least one time regardless of the condition. In this practical, the do-while loop is used to determine whether a number is prime by checking its divisibility with numbers starting from 2.

Syntax:

```
do
{
    // statements
}
while(condition);
```

3. For Loop

The for loop is an entry-controlled loop that combines initialization, condition checking, and updation in a single statement. It is suitable when the number of iterations is known in advance. In this practical, the for loop is used to generate the Fibonacci sequence up to the required nth term by repeatedly adding the previous two numbers.

Syntax:

```
for(initialization; condition; update)
{
    // statements
}
```

Applications

- Reversing numbers and checking palindromes.
- Finding prime numbers for mathematical calculations.
- Generating Fibonacci sequences used in programming, mathematics, and algorithm design.
- Implementing repetitive tasks efficiently in software development.

Advantages of Looping Constructs

- Reduces code redundancy.
- Improves program readability.
- Saves execution time and memory.
- Makes repetitive operations simple and efficient.
- Enhances program maintainability.



Code Snippet:

```
import java.util.Scanner;

public class Practical4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number to reverse: "); // QUESTION 1 :- Reverse of a loop using while loop
        int num = sc.nextInt();
        int temp = num;
        int reversed = 0;
        while (temp != 0) {
            int digit = temp % 10;
            reversed = reversed * 10 + digit;
            temp /= 10;
        }
        System.out.println("Reversed Number: " + reversed);
        System.out.print("\nEnter a number to check prime: "); // QUESTION 2 :- prime number using do-while loop
        int primeNum = sc.nextInt();
        boolean isPrime = true;
        int i = 2;
        if (primeNum <= 1) {
            isPrime = false;
        } else {
            do {
                if (primeNum % i == 0 && primeNum != 2) {
                    isPrime = false;
                    break;
                }
                i++;
            } while (i <= primeNum / 2);
        }
        if (isPrime) {
            System.out.println(primeNum + " is a Prime Number.");
        } else {
            System.out.println(primeNum + " is not a Prime Number.");
        }

        System.out.print("\nEnter the nth term of Fibonacci sequence: "); //QUESTION 3 :- nth term of fibonacci sequence using for loop.

        int fibo = sc.nextInt();
        int first = 0;
        int second = 1;
        int fibonacci;
        System.out.print("Fibonacci Sequence: ");
        for (int j = 1; j <= fibo; j++) {
            System.out.print(first + " ");
            fibonacci = first + second;
            first = second;
            second = fibonacci;
            break;
        }
        System.out.println();
        sc.close(); } }
```



OUTPUT:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\vishal gohil\Desktop\vishal java file> java Practical4
Enter a number to reverse: 1234
Reversed Number: 4321

Enter a number to check prime: 6
6 is not a Prime Number.

Enter the nth term of Fibonacci sequence: 10
Fibonacci Sequence: 0 1 1 2 3 5 8 13 21 34
PS C:\Users\vishal gohil\Desktop\vishal java file> |
```